

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Athisaya U	Reg.No.:	192571001
Department:	B.Tech Computer Science and Biosciences	Date:	

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints: CPU utilization must be below 15%; multiple sensors generate interrupts every 1 ms; use vectored interrupt handling; select between memory-mapped and I/O-mapped I/O for optimal latency.

ANSWER

1. Constraint analysis and priority

The design must guarantee a **1 ms event period**, direct interrupt dispatch, and a strict **CPU utilization below 15%**. Therefore the hard interrupt path must contain only device acknowledgement, one status/data read, timestamping, and buffer insertion. Filtering, logging, and communication are deferred to a lower-priority task.

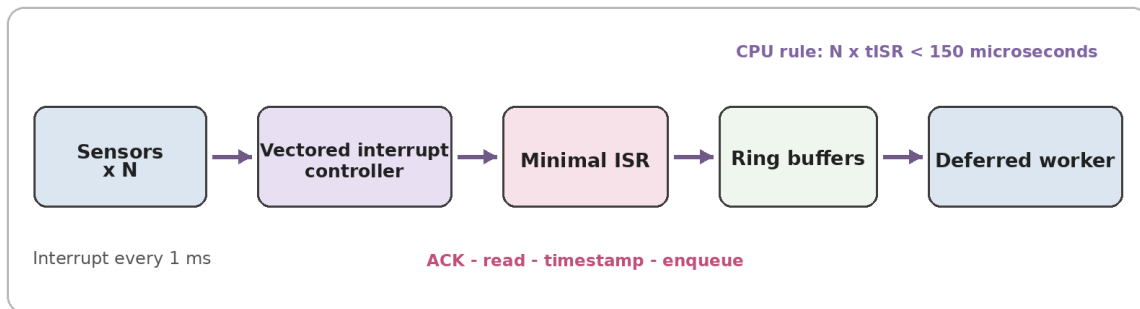


Figure 1. Vectored sensor-interrupt path with bounded ISR work.

Constraint	Priority	Design implication
Interrupt every 1 ms	Highest	Bounded ISR time and deterministic dispatch
CPU utilization < 15%	Highest	Short ISR; defer all noncritical work
Vectored handling	Required	One vector entry per sensor source
Minimum register latency	High	Use memory-mapped device registers

2. Memory-mapped versus I/O-mapped I/O

Feature	Memory-mapped I/O	I/O-mapped I/O
Instruction access	Normal load/store instructions	Special IN/OUT instructions
Addressing	Full processor addressing modes	Separate and usually limited I/O space
Frequent sensor access	Fast and compiler-friendly	Extra instruction restrictions
Selected option	Yes - lower software overhead	No for this latency-critical path

Memory-mapped I/O is selected. Sensor control, status, data, and interrupt-acknowledge registers are placed in a protected address range and declared volatile so every access reaches the hardware.

3. Vectored interrupt architecture

Block	Responsibility	Timing rule
Sensor interfaces	Latch sample and assert individual interrupt request	Every 1 ms
Vectored interrupt controller	Prioritize requests and supply the ISR address	No polling for source
Sensor ISR	Acknowledge, read, timestamp, enqueue, return	Bounded execution
Ring buffers	Hold samples until the worker consumes them	One buffer per source
Deferred worker	Filter, combine, log, and transmit samples	Runs outside hard ISR

4. CPU utilization budget

For N sensors, each interrupting once every 1 ms, the interrupt utilization is $U = (N \times t_{ISR}) / 1 \text{ ms}$. To keep U below 0.15, the implementation must satisfy $N \times t_{ISR} < 150 \text{ microseconds}$. For example, with six sensors the worst-case ISR time per sensor must be below 25 microseconds.

Item	Budget / action	Reason
ISR work	Acknowledge + read + enqueue only	Keeps t_{ISR} bounded
Simultaneous requests	Fixed priorities with rotating order among equals	Protects critical sensors without starvation
Buffer overflow	Watermark and dropped-sample counter	Makes overload visible
Deferred processing	Low-priority scheduled task	Cannot extend hard interrupt latency

5. Testing and validation

Test	Required observation
All sensors interrupt together	Highest-priority vector runs first; equal classes rotate fairly
1,000 consecutive periods	No missed sample and no ring-buffer corruption
Worst-case ISR timing	$N \times t_{ISR}$ remains below 150 microseconds per 1 ms
Background overload	Hard ISR latency remains bounded; overflow counter reports pressure

Result: Vectored dispatch plus memory-mapped registers and very short ISRs satisfies the latency requirement. The explicit timing inequality proves that CPU utilization can be maintained below 15%.

2. Design an I/O subsystem under these constraints: high-speed storage requires throughput > 3 GB/s; must use NVMe over PCIe; CPU should not handle bulk data transfer; integrate DMA controller with interrupt notification.

ANSWER

1. Throughput and interface selection

An **NVMe SSD on PCIe Gen4 x4** is selected. Its link capacity provides comfortable margin above the required 3 GB/s after protocol overhead. The NVMe controller acts as a PCIe bus master and transfers payload directly between the SSD and main memory.

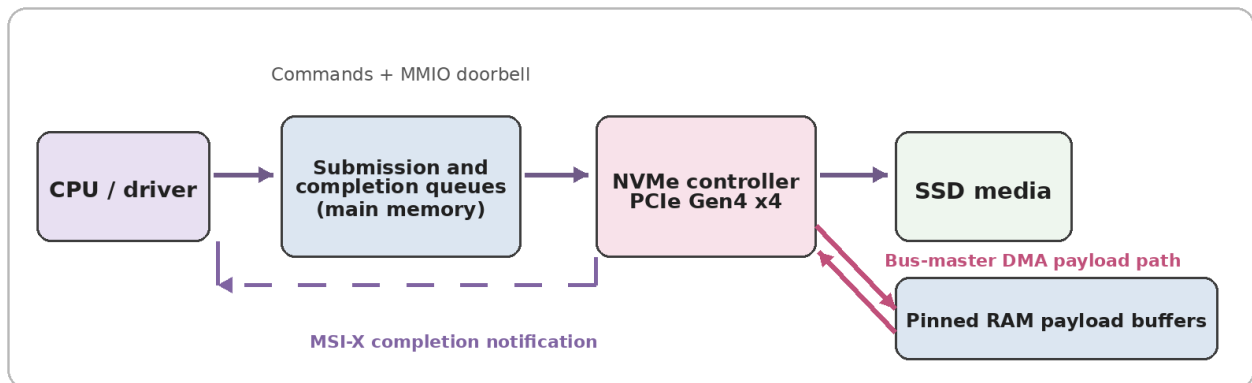


Figure 2. NVMe command, DMA payload, and MSI-X completion paths.

Requirement	Selected design	Verification
Throughput > 3 GB/s	PCIe Gen4 x4 NVMe path	Link capacity exceeds the target with margin
NVMe over PCIe	NVMe submission/completion queues	Native protocol path
No CPU bulk transfer	PCIe bus-master DMA	Payload bypasses CPU copy loops
Interrupt notification	MSI-X completion vectors	Precise per-queue notification

2. Queue and DMA architecture

Stage	Operation
1. Command submission	CPU writes an NVMe command into a submission queue in RAM.
2. Doorbell	CPU writes the memory-mapped queue-tail doorbell register.
3. Descriptor fetch	NVMe controller fetches the command and scatter-gather list through PCIe.
4. Bulk transfer	Controller DMA moves blocks directly between SSD and pinned RAM buffers.
5. Completion	Controller writes a completion entry and raises the assigned MSI-X vector.
6. ISR	ISR acknowledges the vector, consumes completion entries, and wakes the task.

3. DMA controller configuration

1. Allocate **DMA-capable aligned buffers** and map them through the IOMMU.
2. Build scatter-gather descriptors so non-contiguous pages can be transferred without CPU copying.
3. Create separate submission and completion queues; distribute queues across CPU cores.
4. Enable MSI-X and assign completion vectors per queue or queue group.
5. Use interrupt coalescing only for bulk workloads where a small notification delay is acceptable.

4. CPU involvement comparison

Activity	CPU	NVMe controller / DMA
Build command	Yes - small descriptor work	No
Move multi-megabyte payload	No	Yes - direct SSD to RAM transfer
Track completion	Consumes completion entry	Writes entry and raises MSI-X
Error recovery	Driver handles exception	Reports status code

5. Testing and validation

Test	Pass condition
Sequential read/write benchmark	Sustained measured throughput remains above 3 GB/s
CPU profiler during transfer	No memcpy loop on the data path; low CPU utilization
DMA integrity test	Buffer checksum equals stored block checksum
Interrupt test	Exactly the expected completion entries are consumed without lost vectors

Result: NVMe over PCIe Gen4 x4 supplies the required bandwidth. Bus-master DMA performs bulk movement, while MSI-X interrupts notify the CPU only when queue work completes.

3. Construct a multi-device communication architecture with constraints: 6 peripherals using USB, 2 high-speed devices using Thunderbolt; avoid interrupt starvation; use hardware and software interrupts; ensure fair bus arbitration.

ANSWER

1. System topology

The six USB peripherals connect through a powered hub tree to one USB host controller. The two high-speed devices connect through a Thunderbolt controller attached to the PCIe root complex. A common interrupt controller receives hardware events, while software interrupts perform deferred driver work.

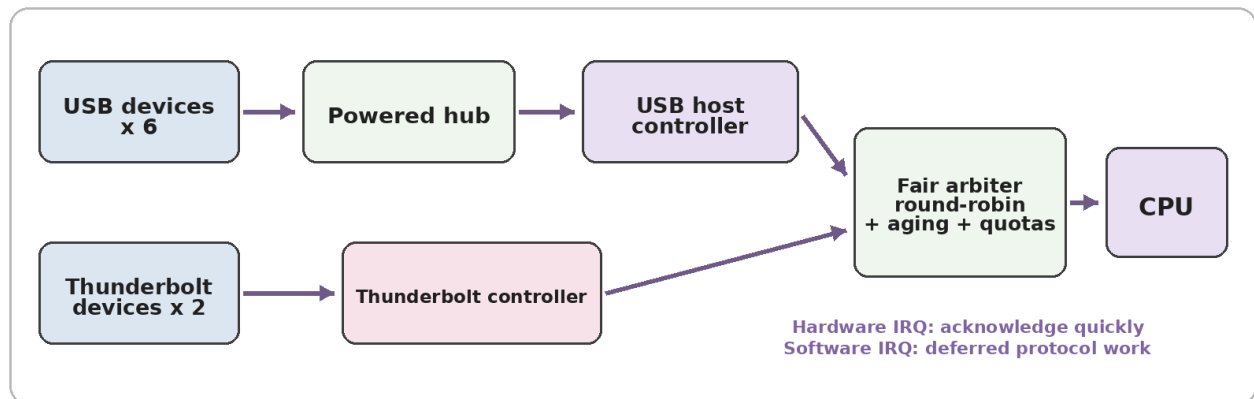


Figure 3. Shared USB and Thunderbolt topology with fair arbitration.

Device group	Interface path	Service class
USB periodic devices	USB device -> powered hub -> USB host controller	Guaranteed periodic service
USB bulk/control devices	USB device -> powered hub -> USB host controller	Round-robin within remaining slots
Thunderbolt devices x2	Device -> Thunderbolt controller -> PCIe fabric	High bandwidth with bounded quota

2. Hardware and software interrupt split

Interrupt level	Work performed	Bound
Hardware ISR	Read cause, acknowledge device, enqueue completion token	Small fixed quota
Software interrupt	Finish protocol processing and schedule client task	Preemptible by hardware IRQ
Background thread	Logging, statistics, retries, nonurgent cleanup	Lowest priority

A hardware ISR must never drain an unlimited queue. After processing a fixed number of completions, it schedules a **software interrupt** and returns. This prevents one busy device from holding the CPU in interrupt context.

3. Starvation prevention

1. Assign urgent periodic USB endpoints the highest bounded service class.
2. Use **round-robin order** among devices in the same class.
3. Apply **priority aging** so a request waiting beyond its limit is temporarily promoted.
4. Limit every ISR to a completion budget, then rotate to the next pending vector.
5. Reserve a minimum share for each Thunderbolt and USB queue before allocating excess bandwidth.

4. Fair bus arbitration

Traffic class	Arbitration rule	Fairness guarantee
USB periodic	Reserved microframe slots	Deadlines met first
USB bulk/control	Round-robin endpoint queues	Every active endpoint advances
Thunderbolt x2	Weighted round-robin PCIe queues	Both devices receive bounded shares
Excess capacity	Work-conserving allocation	Unused share is redistributed

5. Testing and validation

Stress case	Pass condition
Both Thunderbolt devices saturated	USB periodic deadlines still met
Six USB devices active together	No endpoint waiting time exceeds its bound
One device produces continuous IRQs	ISR quota forces rotation to other vectors
Low-priority request held pending	Aging promotes and completes the request

Result: Bounded hardware ISRs, deferred software interrupts, round-robin service, priority aging, and reserved bandwidth prevent starvation while preserving high-speed Thunderbolt performance.

4. Design a data acquisition system with constraints: continuous data stream at 500 MB/s; minimal CPU intervention required; must compare DMA transfer vs interrupt-driven I/O; use memory-mapped I/O for device registers.

ANSWER

1. Data-rate analysis

At 500 MB/s, interrupting for every 4-byte word would generate **125 million interrupts per second**. Even one interrupt per 4 KiB block produces about 122,070 interrupts/s. A scatter-gather DMA engine with 1 MiB buffers needs only about **477 buffer completions per second**.

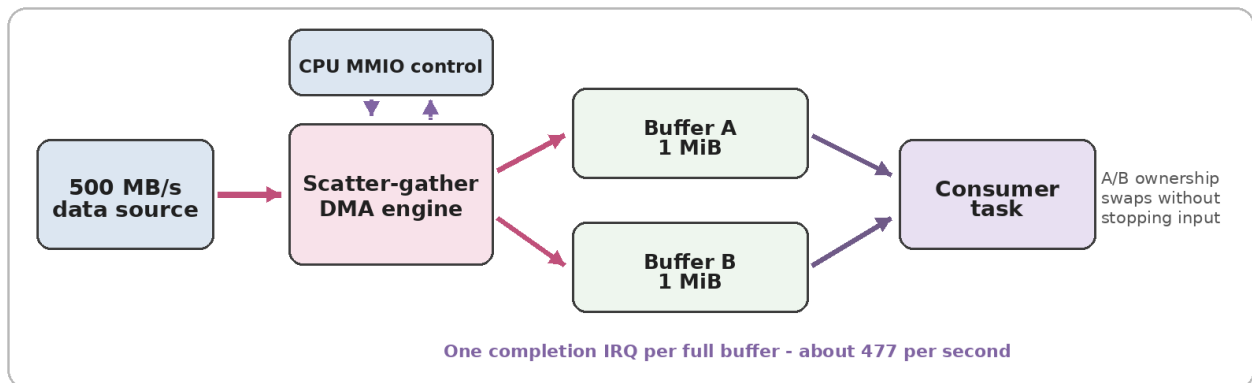


Figure 4. Continuous cyclic-DMA acquisition using alternating buffers.

Transfer granularity	Events per second at 500 MB/s	CPU effect
4-byte word	125,000,000	Impossible for interrupt-driven I/O
4 KiB block	About 122,070	High interrupt overhead
1 MiB DMA buffer	About 477	Very low completion overhead

2. DMA versus interrupt-driven I/O

Feature	Interrupt-driven programmed I/O	DMA transfer
Payload movement	CPU reads/writes each unit or small block	DMA engine moves full buffers
Interrupt rate	Very high at 500 MB/s	One interrupt per buffer/batch
CPU utilization	High and unpredictable	Low and bounded
Continuity risk	Overflow during CPU delay	Descriptor ring absorbs scheduling jitter
Selected method	Not selected for data path	Scatter-gather cyclic DMA

3. Memory-mapped register interface

Register	Purpose
CONTROL	Enable acquisition, start/stop DMA, reset channel
STATUS	Running, buffer-full, error, overrun state
DESC_BASE	Address of the scatter-gather descriptor ring
BUFFER_LENGTH	Bytes assigned to each DMA buffer
IRQ_STATUS / IRQ_ACK	Read cause and acknowledge completion/error
BYTE_COUNT	Monitor sustained transfer and detect loss

4. Double-buffered acquisition sequence

1. Allocate two or more aligned 1 MiB buffers and build a cyclic descriptor ring.
2. Write descriptor base, length, and interrupt-enable values to memory-mapped registers.

3. DMA fills buffer A while the consumer processes buffer B.
4. On completion, the ISR acknowledges the device, records the buffer index, and wakes the consumer.
5. Ownership swaps immediately; the next descriptor is already available, so acquisition never pauses.

5. Testing and validation

Measurement	Pass condition
Sustained byte count	At least 500 MB/s for the full test interval
Overflow/error status	No overflow and zero dropped-buffer count
CPU profiler	Only buffer completion work; no per-word copy loop
Buffer sequence numbers	Continuous sequence with no gap or duplicate

Result: Scatter-gather DMA with large cyclic buffers reduces event rate from millions to hundreds per second. Memory-mapped registers provide fast control while the CPU handles only completion and error notifications.

5. Construct an interrupt handling mechanism with constraints: support nested interrupts; maximum interrupt latency < 10 microseconds; use vectored interrupts for high-priority devices; use software interrupts for background tasks.

ANSWER

1. Priority and vector organization

Priority	Source	Handling
P3 - Critical	Safety/timing device	Dedicated vector; may preempt P2/P1/P0
P2 - High	High-rate I/O completion	Dedicated vector; may preempt P1/P0
P1 - Normal	Ordinary device events	Vectored hardware ISR
P0 - Background	Deferred protocol and maintenance work	Software interrupt / worker

On entry, the controller masks the current and lower priorities but leaves higher priorities enabled. Therefore a P3 event can nest inside a P2 or P1 handler, while equal-priority re-entry is blocked.

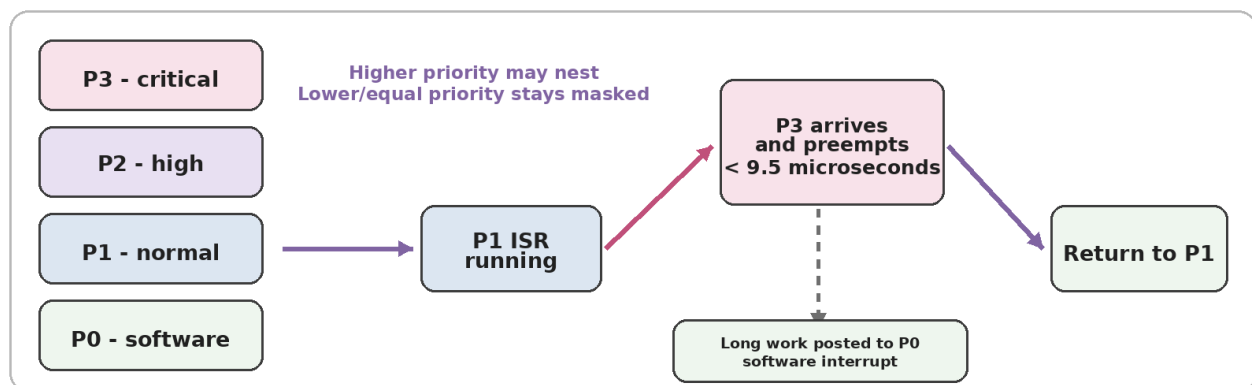


Figure 5. Nested interrupt preemption and deferred software work.

2. Sub-10-microsecond latency budget

Latency component	Maximum budget
Interrupt synchronization and detection	1.0 microsecond
Priority arbitration and vector fetch	1.0 microsecond
Minimal context save	2.0 microseconds
ISR prologue and source acknowledge	2.0 microseconds
First critical device action	2.0 microseconds
Reserved timing margin	1.5 microseconds
Enforced total	9.5 microseconds maximum

The measured path to the first critical device action must be **below 8 microseconds**, retaining 1.5 microseconds of margin while keeping the enforced total strictly below 10 microseconds.

3. Nested ISR sequence

1. Hardware supplies the vector; the CPU saves only the required volatile context.
2. ISR acknowledges the source immediately and raises the active priority threshold.
3. Critical status is captured into a preallocated per-device record.
4. A higher-priority vector may preempt and use its own reserved stack space.
5. The ISR posts a software interrupt for longer processing and returns.
6. The software interrupt runs only when no higher-priority hardware event is pending.

4. Correctness and safety controls

Risk	Control
Stack overflow during nesting	Bound maximum nesting depth and reserve stack per level
Shared-data corruption	Use atomic flags or very short priority-ceiling critical sections
Long ISR execution	No allocation, logging, blocking call, or bulk processing in hard ISR
Background starvation	Software-interrupt quota plus scheduled worker with aging
Lost interrupt	Read cause before acknowledge; verify pending state before return

5. Testing and validation

Test	Pass condition
P3 arrives during P1 ISR	P3 preempts and reaches first critical action in < 10 microseconds
Maximum nesting depth	No stack corruption; correct return order
Continuous P2 traffic	P0 software work eventually runs without delaying P3
Worst-case bus load	Latency analyzer still reports < 10 microseconds

Result: Dedicated vectors, bounded nesting, minimal context save, and deferred software interrupts provide predictable priority handling. The explicit 10-microsecond budget makes the latency constraint directly testable.

Code of Conduct:

I Athisaya U / 192571001 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided